FundGoodDeeds / RELEASE 1

Project Design Document

TEAM 3C JPY

Jun Zheng <jz7259@rit.edu>

Evan Ke <ek6747@rit.edu>

Kevin Zheng <kz3175@rit.edu>

Google Doc link to the document with version history:

Release 1: SWEN-383-DesignDoc-3C.docx

Table Of Contents

Table Of Contents 2
1 Project Summary 3
2 Design Overview 3
Process 3
Discover 3
Requirements 3
Constraints 4
Guidelines 4
Metrics of Success 4
Define 5
Development 12
Delivery 12
Challenges Faced 12
What we learned & Successes 12
3 Subsystem Structure 14
4 Subsystems 15
4.1 Default Subsystem 15
4.2 Model Subsystem 16
4.3 View Subsystem 19
4.4 Controller Subsystem 20
5 Sequence Diagrams 21
5.1 Sequence Diagram #1 — Creating need 21
Purpose 21
Scenario and Feature 21
Process Explanation 21
5.2 Sequence Diagram #2 — Read Needs + Update 22
Purpose 22
Scenario/Feature 22
Process Explanation 23
5.3 Sequence Diagram #3 — Read Needs + Update 23
Purpose 23
Scenario/Feature 23
Process Explanation 24
6 Pattern Usage 25
6.1 Pattern #1 - Composite Pattern 25
6.2 Pattern #2 - MVC Pattern 25
6.3 Pattern #3 - Observer Pattern 25
7 RATIONALE 26

1 Project Summary

The Community Empowerment Consortium is funding the creation of a financial resilience tracking program. The software will help users monitor and improve their finances through daily tracking and goal-setting.

Users can define basic needs (e.g., housing, transportation, healthcare) with details like total, fixed, variable costs, and fees. These needs can be combined into bundles for specific situations. Users can also create custom needs and bundles.

Every day, users record ledger entries showing which needs or bundles were fulfilled and their quantities. The program will calculate daily totals and generate visualizations showing the spending breakdown across fixed, variable, and fee categories.

Users can also log their available funds and daily funding goals, allowing the program to indicate whether they are on track, underfunded, or in surplus.

All data—including defined needs, bundles, and daily ledger entries—must be saved and persisted for future use.

2 Design Overview

Process

Discover -> Define -> Development -> Delivery -> Repeat based on feedback

1. In the discovery phase, we figured out all project requirements, constraints, guidelines, and what we defined as a “metric of success”. We also researched what patterns we should use based on these constraints and requirements.
2. In the define phase, we utilized our knowledge from discovery to create initial class and sequence diagrams. These were quick diagrams hand-sketched by group members, with the intention of combining them into one finalized, definitive “answer” for development. Through iteration, we finalized what we would define as the fundamentals ready for development.
3. In the development phase, we split up the work into functionality, with each person doing one part: Needs, Bundles, and Entry. Each member worked through their deliverable, and the code was merged when completed. Throughout development, we utilized standups to ensure that progress was moving along smoothly without blockers.
4. Once development was complete, we packaged everything into what we defined as our metric of success, or the finished project for phase 1 of this project.
5. Based on our submission and feedback, we would go back to each step to iterate and perfect our work based on any possible improvements we can implement.

Discover

Requirements

Firstly, we had to figure out what the project was about - a financial tracking and goal-setting application that helps users monitor their daily spending and progress toward funding goals. We analyzed the requirements to understand how needs, bundles, and daily ledger entries interact.

Constraints

Secondly, we had to figure out project constraints. What were the parts of the assignments we *had* to follow? Upon inspection, we discovered that:

- ☐ We had to utilize two patterns: The MVC and the composite pattern
- ☐ We had to follow a CSV-based storage system
- ☐ We had to apply good design principles, such as SOC, to ensure a good grade.
- ☐ We had a time limit based on the given project timeline

Guidelines

With the constraints given to us, we created guidelines that we had to follow throughout the project. This ensured that we met project requirements in a timely manner. These included:

- ☑ We will utilize both the MVC and composite patterns
- ☑ We will follow a CSV-based storage system for loading and saving data
- ☑ We will start with drawn-out diagrams (class, sequence, subsystem) to make sure that we have a good blueprint and understanding of what we have to develop.
- ☑ We will utilize standups and split up work to ensure that we meet deliverable timelines.
- ☑ We will split our project into three layers (MVC) to follow good design principles. We will utilize abstraction to make sure that we follow principles such as high cohesion and low coupling.

Metrics of Success

Once this was done, we figured out what we considered metrics of success. What would be considered a “successful project”? Upon discussion, we concluded that in order for our project to be successful, we have to:

- ☑ Follow all project requirements
- ☑ Complete everything before the deliverable due dates
- ☑ Be prepared to talk about the parts we did for the project
- ☑ Learn and utilize the design patterns we learned in class

Firstly, we have to get context on what is required for this application. In this case, it is the needs, the daily ledgers, and bundles. We can see that the daily ledgers encapsulate needs and bundles, with bundles grouping needs together. Given this situation, the composite pattern is utilized.

To separate the logic between different layers of the application (separation of concerns), MVC is used to separate data logic, business logic, and client view.

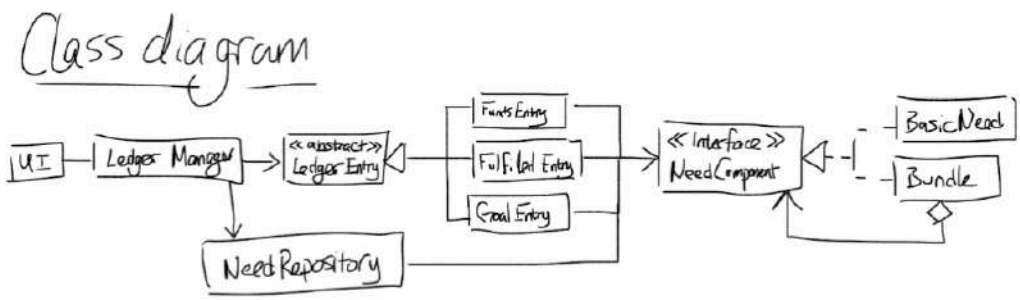
Utilizing both the composite and MVC patterns makes sure there's proper separation of concerns, high cohesion, low coupling, and flexible design.

Define

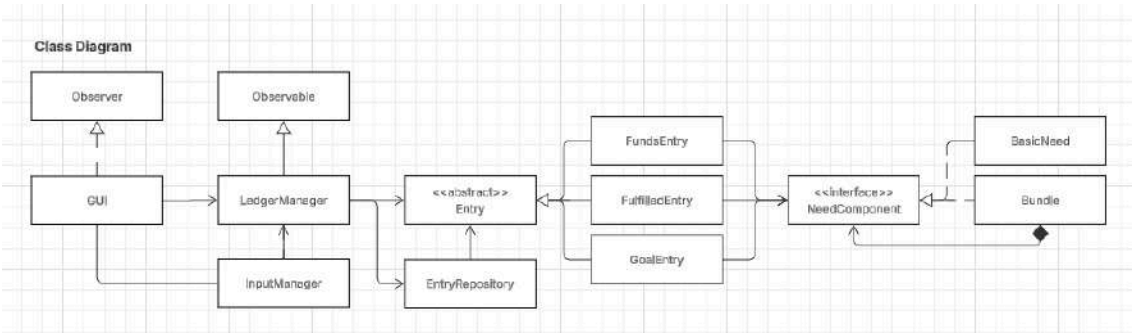
When defining, we decided to take a cooperative and iterative approach. We started by making initial versions of diagrams needed for the development phase, and iterated on them based on feedback given to us.

Class Diagrams

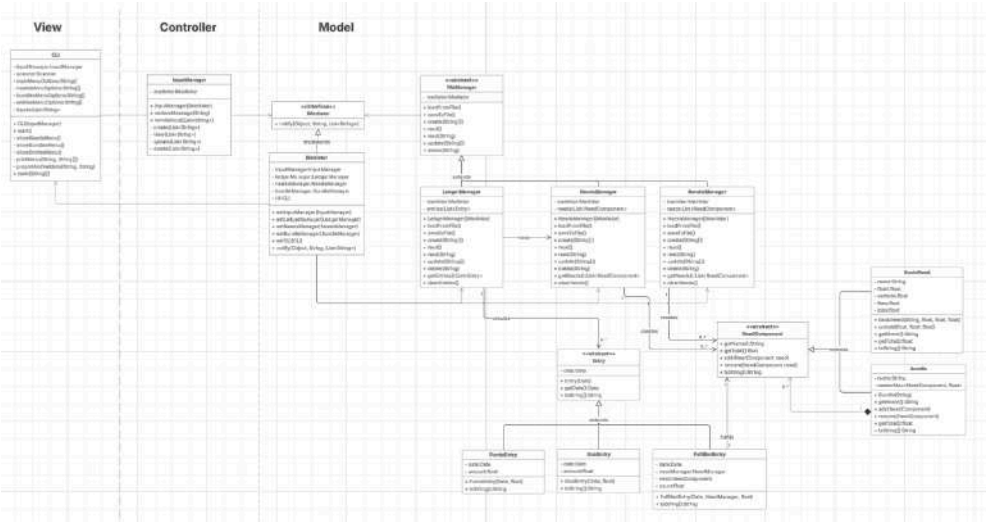
Version 1: *This was just to get a better understanding of how the entire project structure would look - mainly showing class names without functionality.*



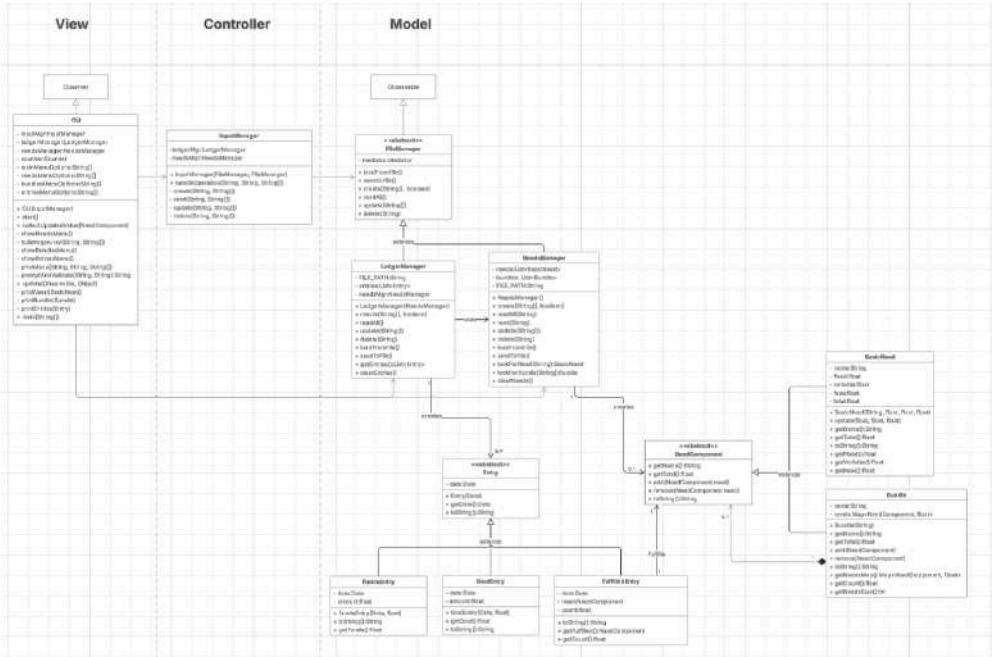
Version 2: *The next iteration. It still doesn't have functionality, but it is a big step up from the first phase, with new classes made for better cohesion and SOC, such as the InputManager.*



Version 3: *This version starts fleshing out the previous version. Through peer and professor feedback, we adjusted class formatting and gave each class detailed functionality, and set the blueprint latest version.*



Version 4: *Our latest version. Further feedback made us adjust and refactor functionality. For example, we decided to adjust to an observer pattern rather than a mediator to follow better design principles.*



Finalized Class Diagram Explanation: We separated our diagram into three sections to adhere to the requirements: View, Controller, and Model. The view contains the CLI, the view for the user. The CLI shows menus and outputs based on what the user wants to see. Next, the controller. The controller acts as the communicator and midpoint for the view and model. The view interacts with the controller, and based on view information, the controller communicates with the model, which contains an abstract FileManager. This FileManager communicates with the Ledger or Needs Manager based on the user's needs.

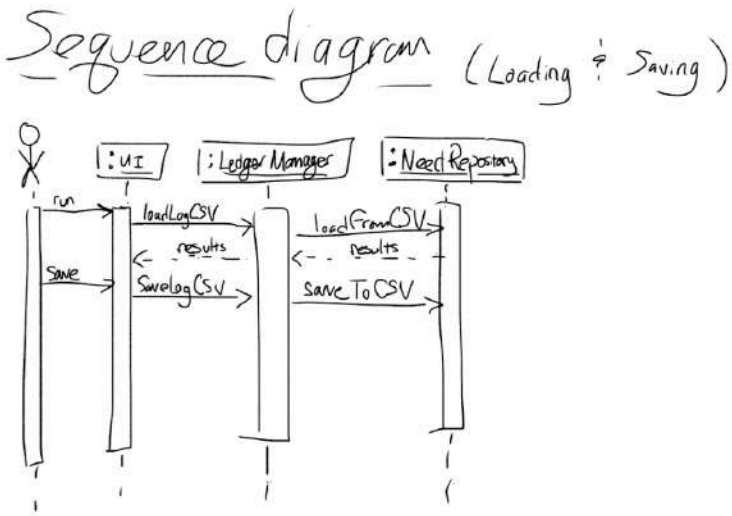
Firstly, the LedgerManager. The LedgerManager tracks and creates all entries (funds, goals, fulfillments). Based on user need, it interacts with an abstract entry, which interacts with either a fundsentry, goalsentry, or fulfillment entry.

Secondly, the NeedsManager. The NeedsManager tracks and creates needs and bundles. Based on user input, it either communicates with the BasicNeed or Bundle class.

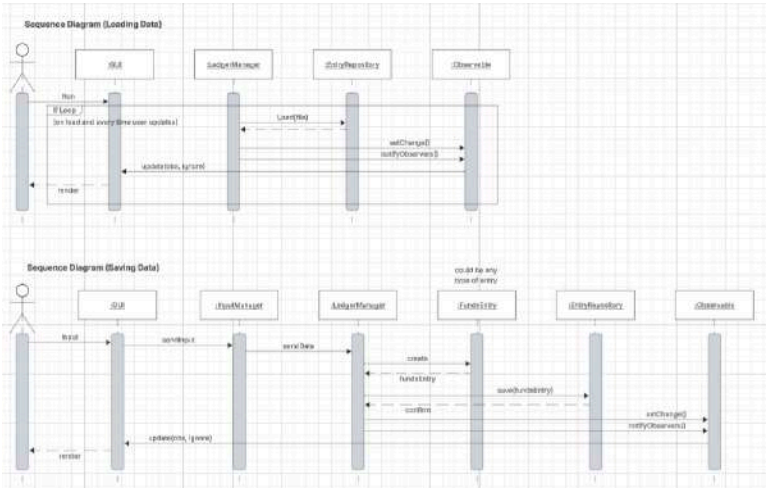
Sequence Diagrams

We followed a similar, iterative process for sequence diagrams. We decided to make flows off different user interactions to see how both the user and application actively work based on different scenarios. **To learn more about our sequence diagrams, *reference our sequence diagram chapter*.**

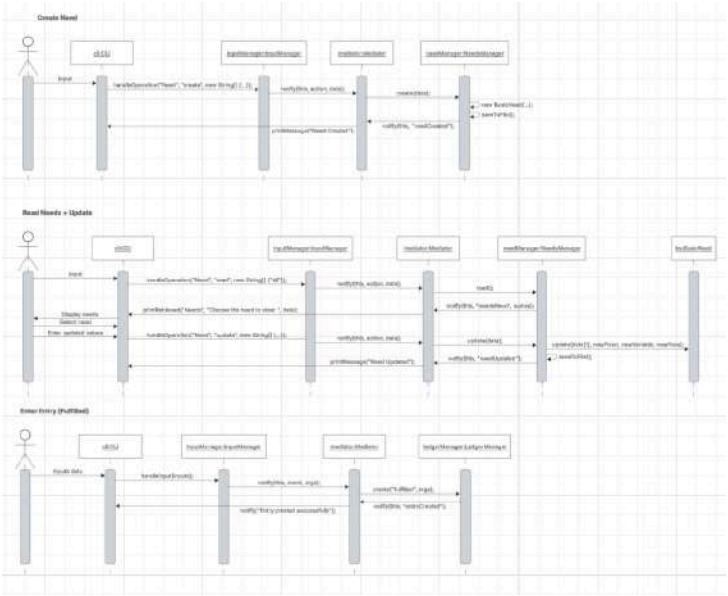
Version 1: *The initial sketches. This was just to understand how the overall loading and saving functions would work*



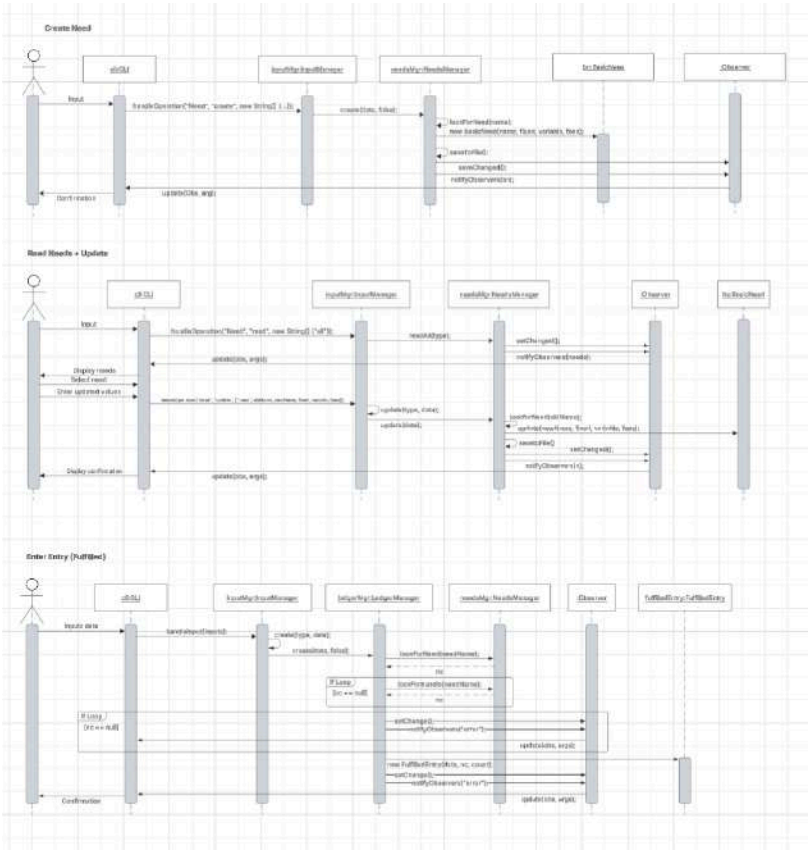
Version 2: *In the second iteration, we just separated the two functionalities (loading and saving) into two diagrams.*



Version 3: *The continued version of the diagrams adds more flows - The Create Need, Reading Needs, and Entry flows.*



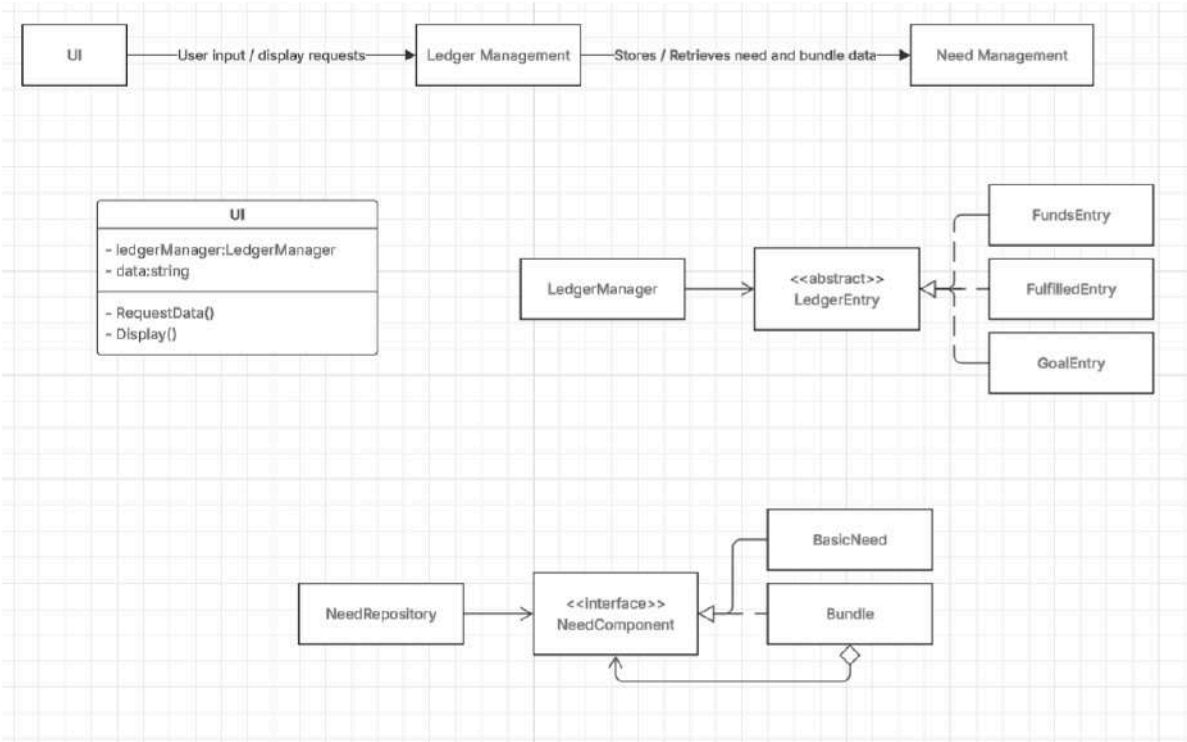
Version 4: *Our final version. Based on peer and professor feedback, we changed the three diagrams from version 3 to better fit our needs.*

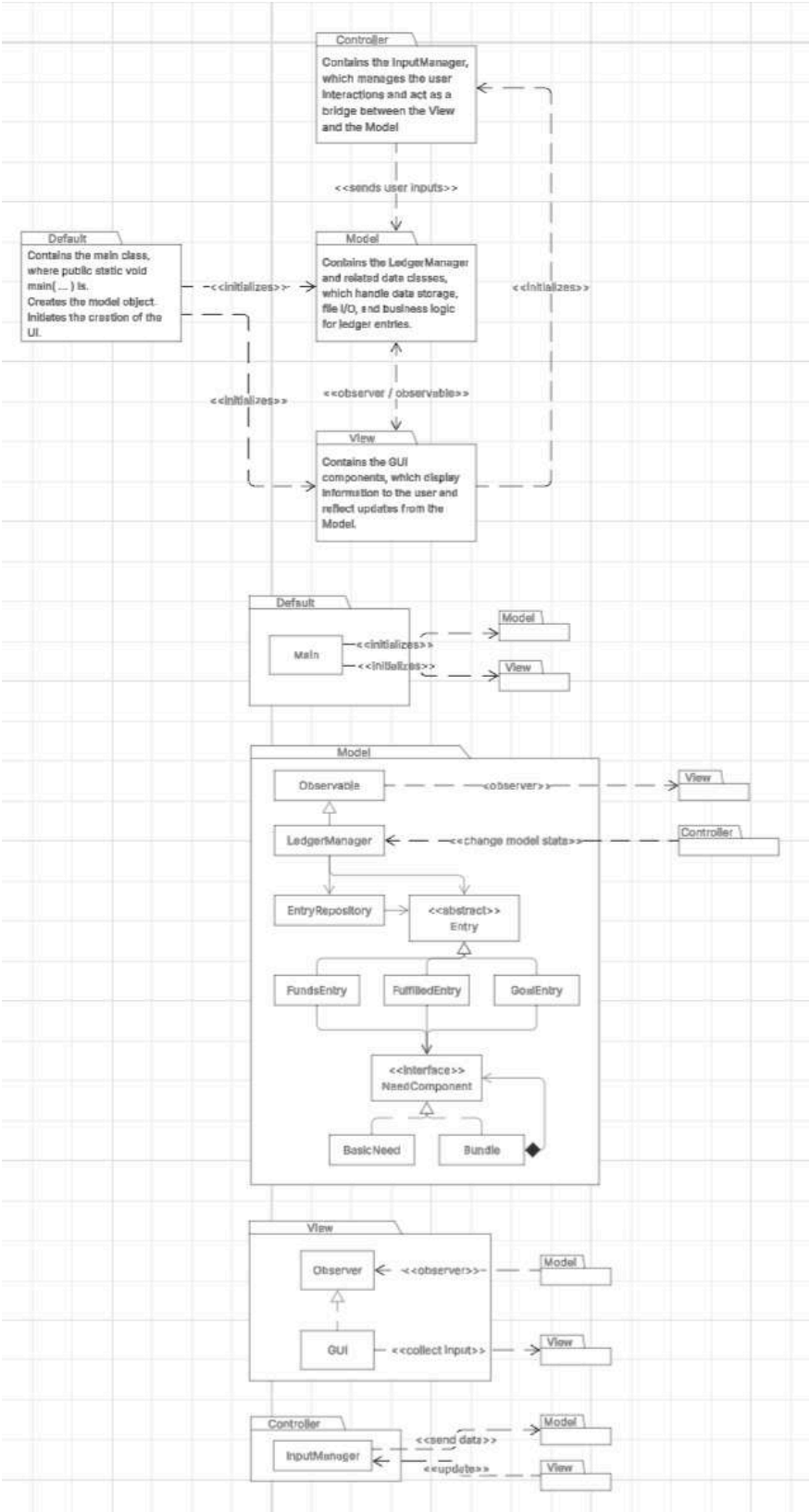


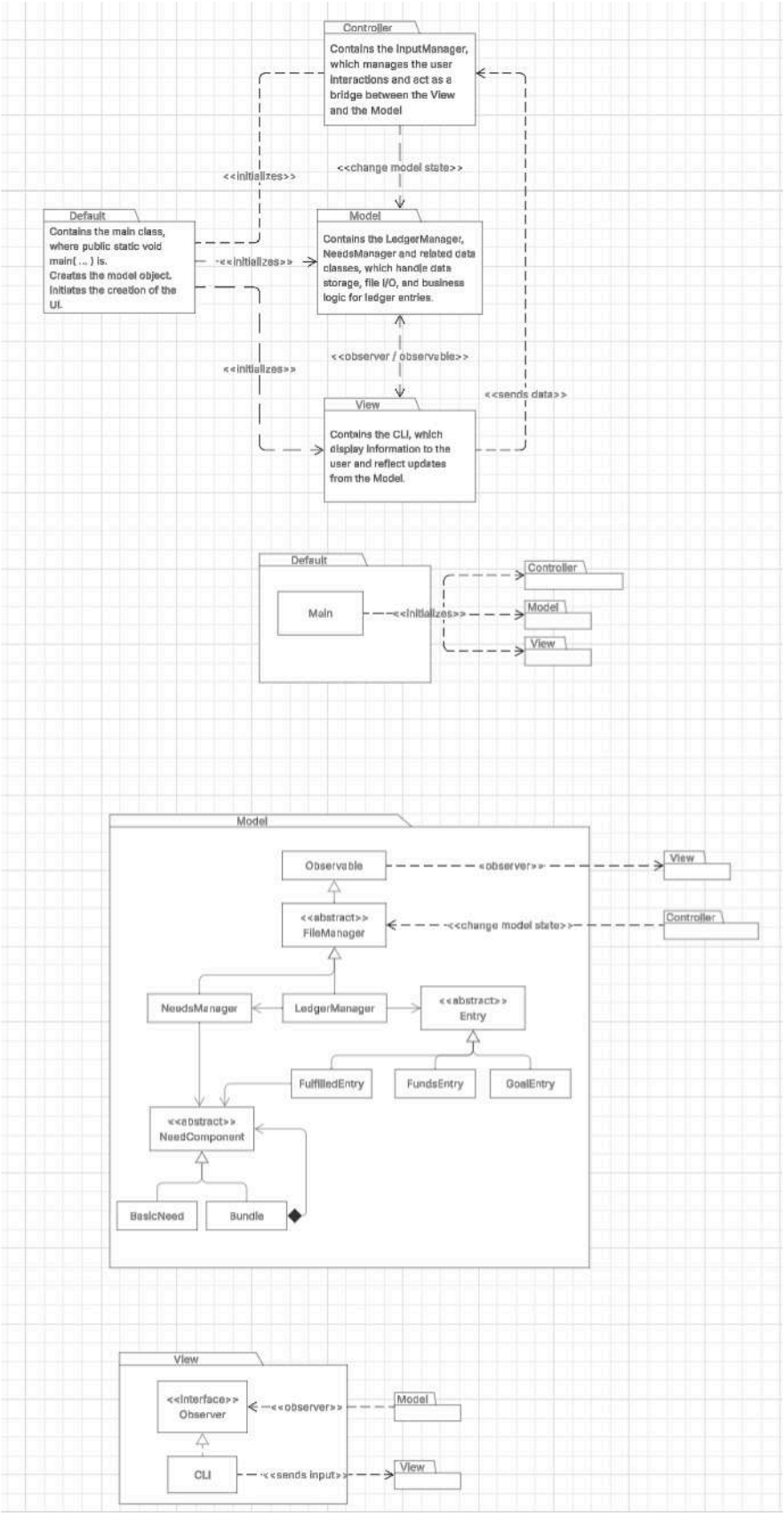
Subsystem Diagrams

The subsystem diagram was made to figure out how each class interacted and connected. **To learn more about our subsystem diagrams, *reference our subsystems chapter*.**

Version 1




Controller
Contains the InputManager, which manages the user interactions and act as a bridge between the View and the Model
<<sends user inputs>>
Default
Contains the main class, where public static void main(...) is.
Creates the model object.
Initiates the creation of the UI.
<<initializes>>
Model
Contains the LedgerManager and related data classes, which handle data storage, file I/O, and business logic for ledger entries.
<<initializes>>
<<initializes>>
<<observer / observable>>
View
Contains the GUI components, which display information to the user and reflect updates from the Model.
Default
Main
<<initializes>>
<<initializes>>
Model
View
Model
Observable
<<observer>>
View
LedgerManager
<<change model state>>
Controller
EntryRepository
<<abstract>>
Entry
FundsEntry
FulfilledEntry
GoalEntry
<<interface>>
NeedComponent
BasicNeed
Bundle
View
Observer
<<observer>>
Model
GUI
<<collect input>>
View
Controller
InputManager
<<send data>>
<<update>>
Model
View


Controller
Contains the InputManager, which manages the user interactions and act as a bridge between the View and the Model
<<initializes>>
<<change model state>>
Default
Contains the main class, where public static void main(...) is.
Creates the model object.
Initiates the creation of the UI.
<<initializes>>
Model
Contains the LedgerManager, NeedsManager and related data classes, which handle data storage, file I/O, and business logic for ledger entries.
<<observer / observable>>
<<initializes>>
<<sends data>>
View
Contains the CLI, which display information to the user and reflect updates from the Model.
Default
Main
<<initializes>>
Controller
Model
View
Model
Observable
<<observer>>
View
<<abstract>>
FileManager
<<change model state>>
Controller
NeedsManager
LedgerManager
<<abstract>>
Entry
FulfilledEntry
FundsEntry
GoalEntry
<<abstract>>
NeedComponent
BasicNeed
Bundle
View
<<Interface>>
Observer
<<observer>>
Model
CLI
<<sends input>>
View

Development

Once the diagrams were all completed, we moved on to development. We did our project in the 383 Github Repo, and divided the work accordingly:

- ☑ Kevin works on the needs functionality.
- ☑ Evan works on the bundle functionality.
- ☑ Junheng works on the entry functionality.
- ☑ The group works collaboratively on the rest of MVC.

To test the project yourself, you can *access the GitHub repository here.*

Delivery

Once development was done, we split delivery into three stages to ensure we met what we considered our metric of success:

1. Presentation: A presentation to talk about our process, work, and challenges. **You can *view the presentation here***
2. Revision: Based on feedback, we went back to revise and iterate on our work. Some of the design processes have been modified and adjusted based on the feedback.
3. Submission: Submission to the phase 1 Dropbox.

Challenges Faced

Throughout the project, we faced my challenges. This included:

- ☑ Working through assignment ambiguities.
- ☑ Figuring out how we should split the work given our constraints. This included:
  - Time Limits from deliverables.
  - Small Group Size (3 as opposed to 4)
  - Learning about new constraints (New patterns we never dealt with)

What we learned & Successes

By facing these challenges, we learned many new lessons in our process of achieving a successful project.

Collaboration

We had to work together in order to get all the functionality done. We each did one part and merged the changes later on once we finished.

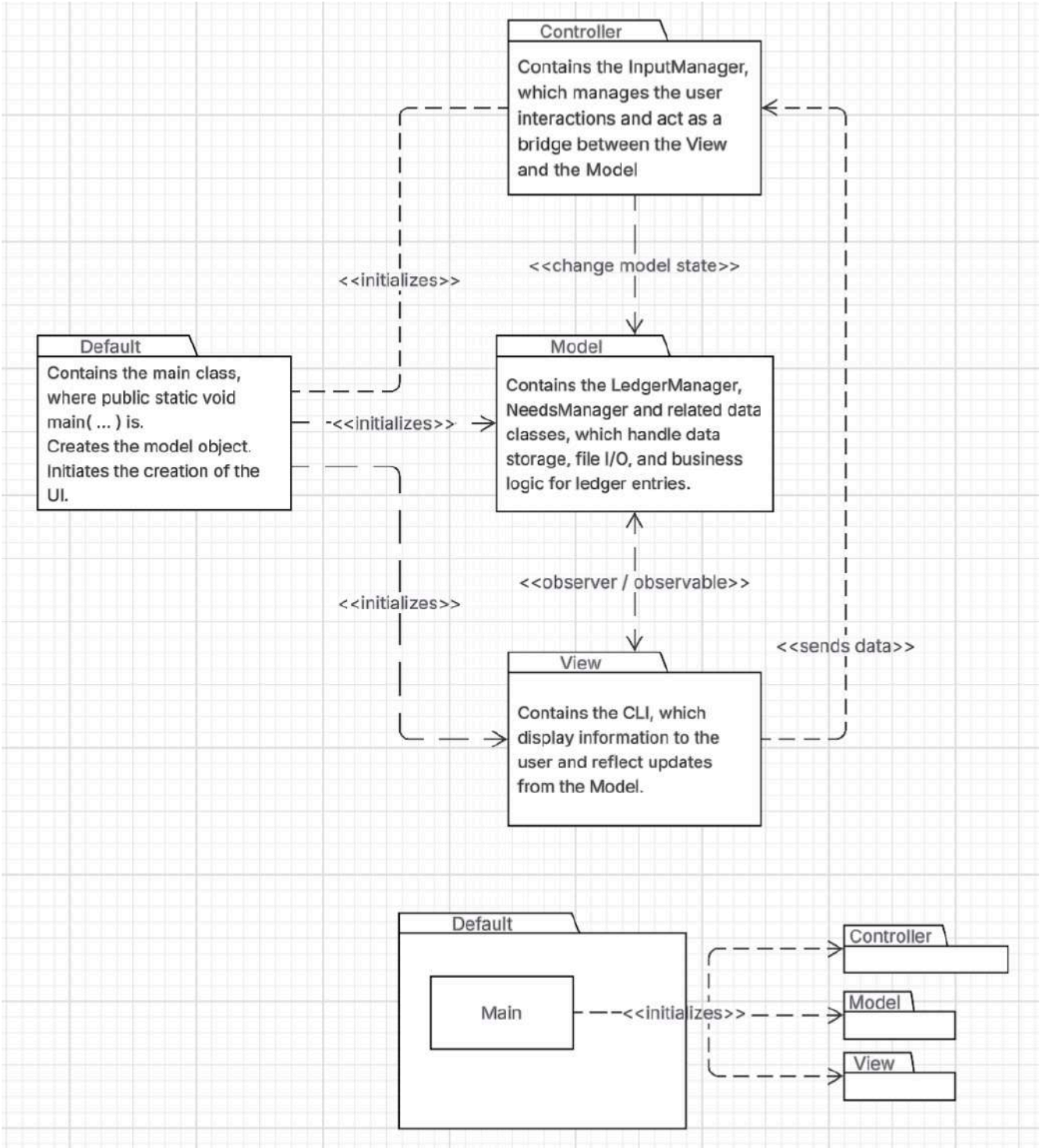
Utilizing Design Patterns

Throughout the project, we had to use design patterns such as the composite and MVC patterns. Learning about these patterns helped us get better at understanding why design patterns are necessary for making scalable and reliable software.

Time Management

This phase of the project had many deadlines and deliverables. We had to collaborate and work together to ensure that we planned and worked on the assignment in a timely manner.

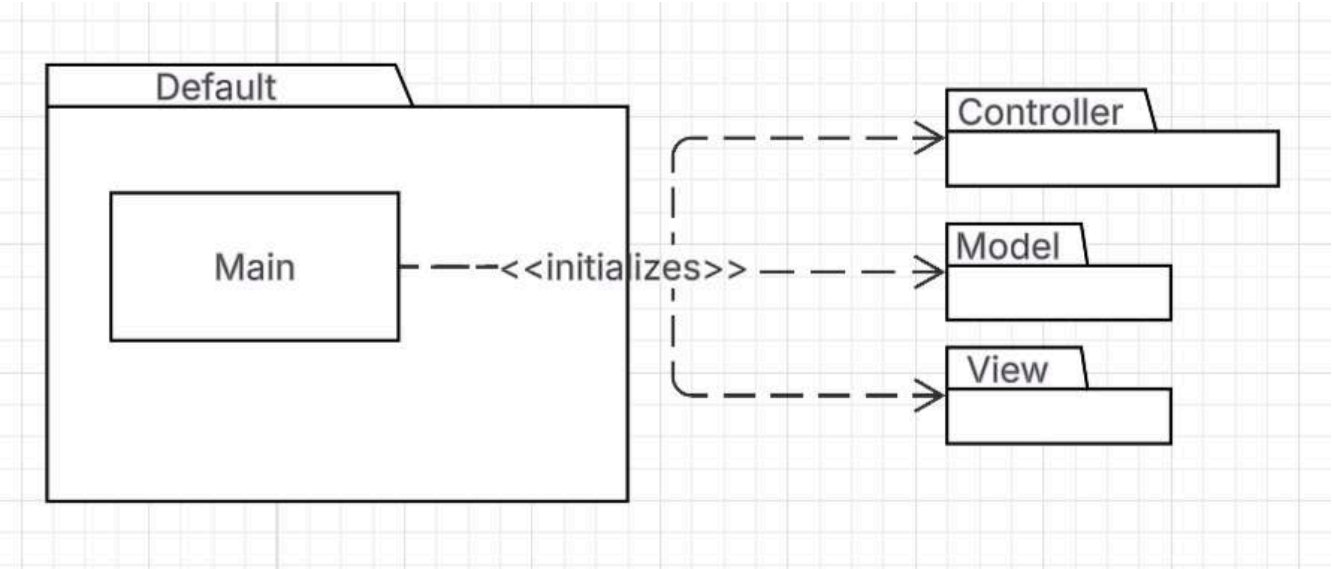
3 Subsystem Structure



4 Subsystems

4.1 Default Subsystem

Class Main	
Responsibilities	Create the model object(s). Create the Controller object. Create the UI object. Connect the subsystems together. Start the UI object.
Collaborators (uses)	**Model.NeedsManager** - the class responsible for maintaining the needs and bundles. **Model.LedgerManager** - the class responsible for maintaining the entries. **Controller.InputManager** - the class responsible for changing model states. **View.CLI** - the primary view class.



4.2 Model Subsystem

FileManager, LedgerManager, NeedsManager, Entry, FundsEntry, FulfilledEntry, GoalEntry, NeedComponent, BasicNeed, Bundle.

Class FileManager (abstract)	
Responsibilities	Define abstract methods for managing the storage of data. Provide a common interface for child classes to implement file operations. Handle saving and loading of entries, needs, or other objects in a structured format (CSV). Extends from the observable in order to be observed.
Collaborators (inherits)	**java.util.Observable** - So file managers can be observed by the View.

Class LedgerManager	
Responsibilities	Manage all ledger-related operations, including maintaining a collection of Entry objects. Interact with the NeedsManager to check if a need or bundle exists. Notify its observers when a request is completed.
Collaborators (inherits)	**Model.FileManager** - Base operations for file management.
Collaborators (manages)	**Model.Entry** - Provide an abstraction layer for all entry objects.
Collaborators (interacts)	**Model.NeedsManager** - Handle storing and retrieving needs and bundles from files.

Class Entry (abstract)	
Responsibilities	Provide a generic class for all entries in the ledger. Provide the name and date of the entry.

Class FundsEntry	
Responsibilities	Represent an entry recording available funds. Provide the amount of funds on a given date.
Collaborators (inherits)	**Model.Entry** - Provide an abstraction layer for all entry objects.
Collaborators (uses)	**Model.NeedComponent** - Define a base class for all need components.

Class FulfilledEntry	
Responsibilities	Represent a record of a fulfilled need. Provide the specific NeedComponent fulfilled and the amount fulfilled.

Collaborators (inherits)	**Model.Entry** - Provide an abstraction layer for all entry objects.
Collaborators (uses)	**Model.NeedComponent** - Define a base class for all need components.

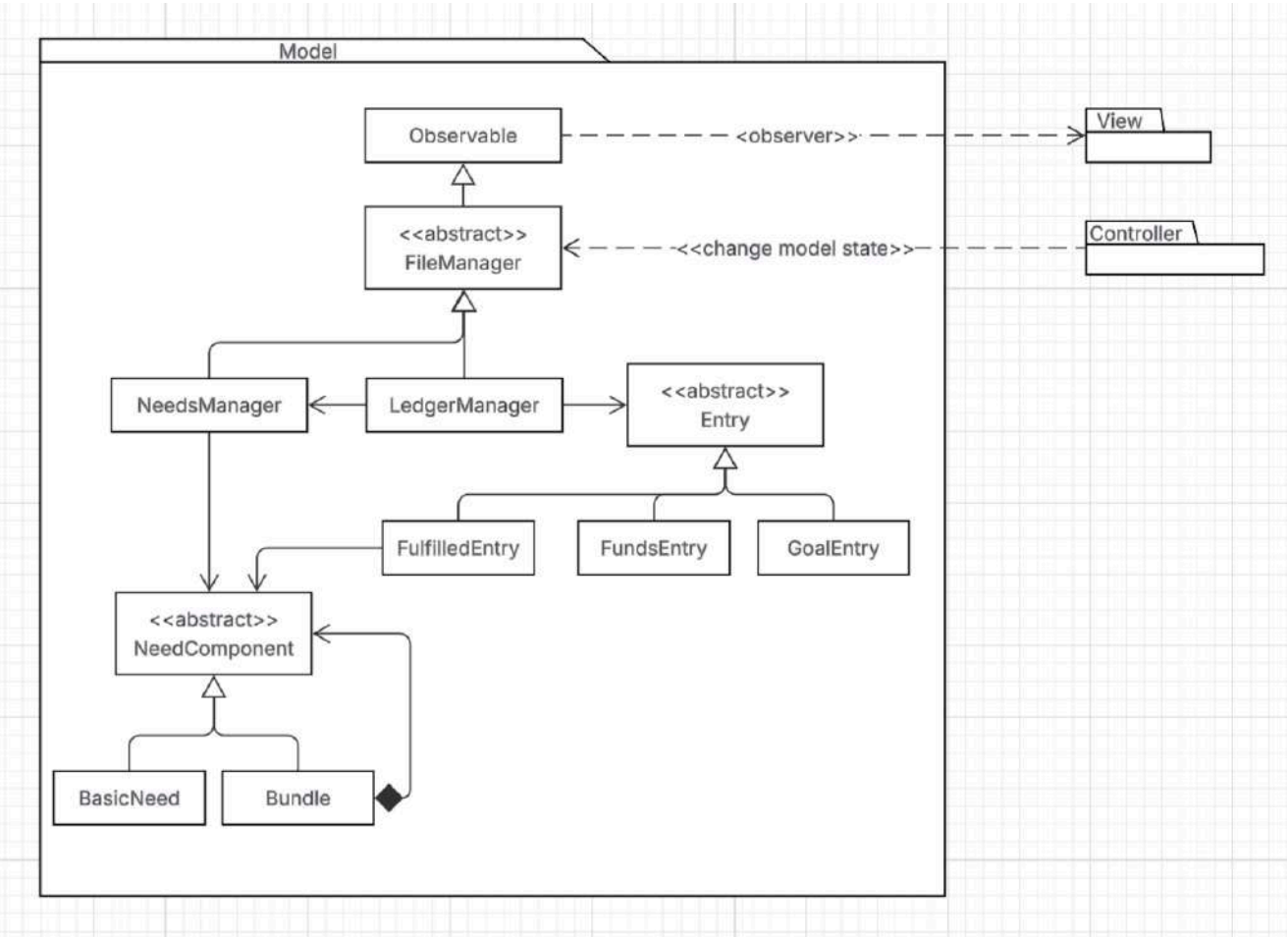
Class GoalEntry	
Responsibilities	Represent an entry recording a goal target. Provide the goal amount at a specific date.
Collaborators (inherits)	**Model.Entry** - Provide an abstraction layer for all entry objects.
Collaborators (uses)	**Model.NeedComponent** - Define a base class for all need components.

Class NeedsManager	
Responsibilities	Manage all needs-related operations, including maintaining a collection of BasicNeed and Bundle objects. Notify its observers when a request is completed.
Collaborators (inherits)	**Model.FileManager** - Base operations for file management.
Collaborators (uses)	**Model.NeedComponent** - Define a base class for all need components.

Class NeedComponent (abstract)	
Responsibilities	Provide a base class for the need and bundle. Provide name and date.

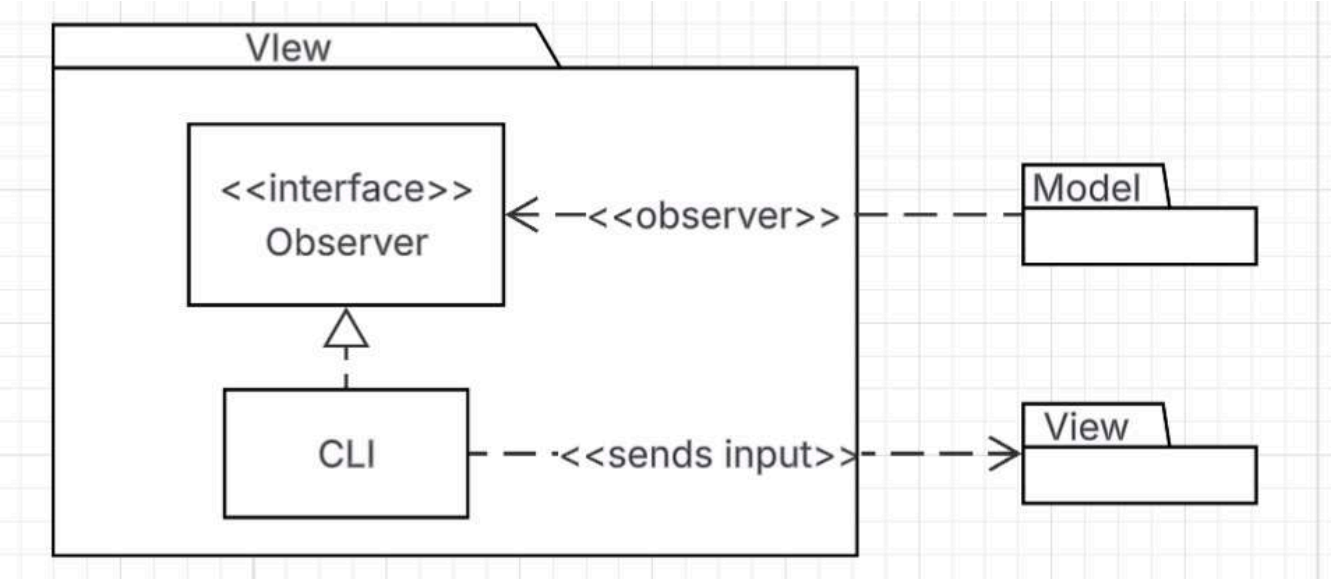
Class BasicNeed	
Responsibilities	Represent a single, fundamental need with associated costs. Provide name, total, fixed, variable, and fees.
Collaborators (implements)	**Model.NeedComponent** - Define a base class for all need components.

Class Bundle	
Responsibilities	Represent a group of needs bundled together. Provide name and list of NeedComponent objects. Add a new NeedComponent. Remove existing NeedComponent.
Collaborators (implements/composes)	**Model.NeedComponent** - Define a base class for all need components.


Model
Observable
<observer>>
View
<<abstract>>
FileManager
<<change model state>>
Controller
NeedsManager
LedgerManager
<<abstract>>
Entry
FulfilledEntry
FundsEntry
GoalEntry
<<abstract>>
NeedComponent
BasicNeed
Bundle

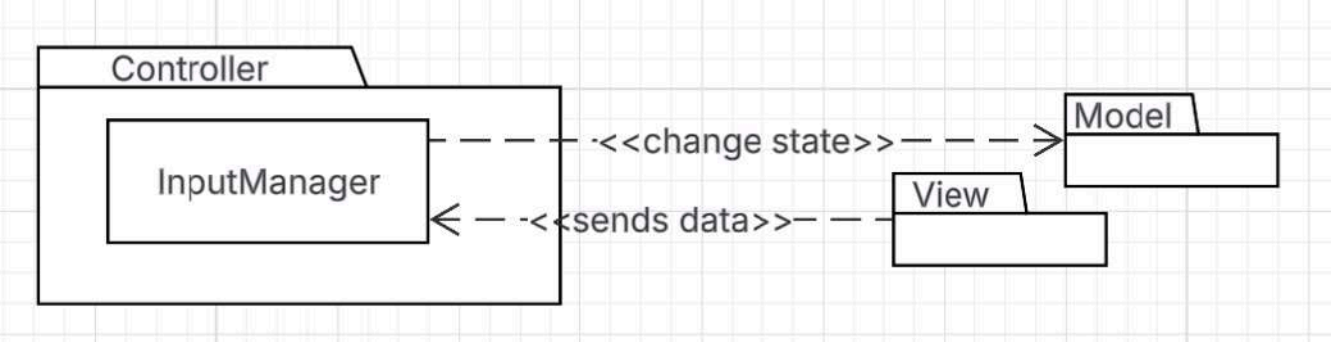
4.3 View Subsystem

Class CLI	
Responsibilities	Route user commands to InputManager for handling operations (create, read, update, delete). Observe changes in LedgerManager and NeedsManager and display updates. Format and print details of Needs, Bundles, and Entries. Handle workflows for updating or deleting items interactively.
Collaborators (implements)	**java.util.Observer** - To register for LedgerManager's and NeedsManager's update notifications.
Collaborators (observes)	**Model.LedgerManager** – Observed for ledger entry changes and updates. **Model.NeedsManager** – Observed for needs and bundles changes.
Collaborators (uses)	**java.util.Scanner** – Collects input from the console. **Controller.InputManager** - Executes operations requested by the user.



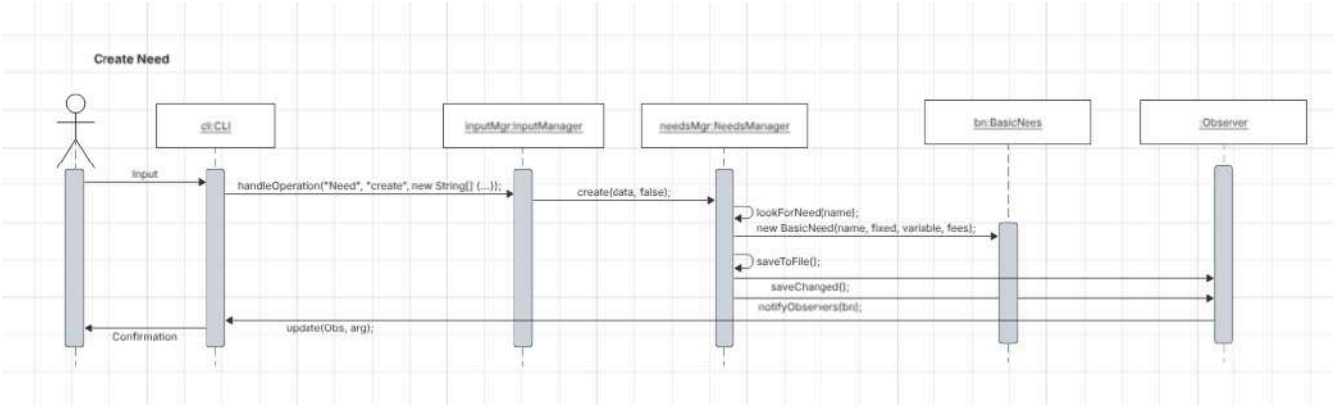
4.4 Controller Subsystem

Class InputManager	
Responsibilities	Receive operation requests from the CLI (e.g., create, read, update, delete). Validate and process input arguments from the CLI. Call the appropriate methods on model classes (LedgerManager, NeedsManager, etc.). Handle data transformation between CLI input and model objects.
Collaborators (uses)	**Model.LedgerManager** – for managing entries. **Model.NeedsManager** – for managing needs and bundles.
Collaborators (receives data from)	**View.CLI** – receives commands from the user interface.



5 Sequence Diagrams

5.1 Sequence Diagram #1 — Creating need



Purpose

This sequence diagram shows how the system handles the creation of a new *Need* (such as food, clothing, etc.) when requested by the user.

It focuses on how input travels through the CLI, is processed by the InputManager, and then stored through the NeedsManager, which triggers updates to observers (such as a UI or log).

The goal is to create a new BasicNeed object and save it, while ensuring all observers are notified of the new data.

Scenario and Feature

A user interacts with the CLI and requests to create a new need (e.g., "create need food fixed 10").

The system accepts the user's input, validates it, creates a new BasicNeed object, stores it, and notifies any observers (like a UI or output screen) that new data is available.

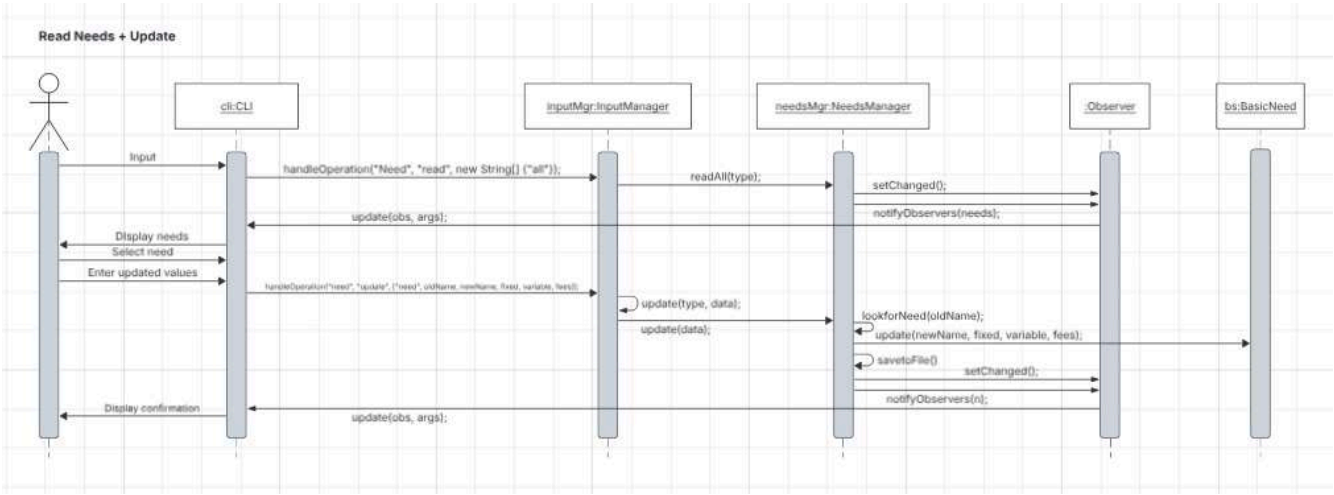
This ensures that once a need is created, all parts of the system that rely on need data are updated automatically.

Process Explanation

1. User -> CLI
    a. The user enters a command.
    b. The CLI captures this as raw input
2. CLI -> InputManager
    a. The CLI forwards the command to InputManager via handleOperation("Need", "create", new String[]{...}).
3. InputManager -> NeedsManager
    a. InputManager parses the command and calls create(data, false) on NeedsManager.
4. NeedsManager -> BasicNeed
    a. NeedsManager checks if the need already exists lookforNeed(name).
    b. If not found, it creates a new BasicNeed(name, fixed, variable, fees) object

5. NeedsManager -> Storage
    a. The new need is saved to a file using saveToFile().
6. NeedsManager -> ObserverPattern
    a. saveChanged() marks the model as updated.
    b. notifyObservers(bn) sends the new BasicNeed object to all observers.
7. Observer -> CLI(or GUI)
    a. Observers receive the update and refresh their display/output.
    b. CLI prints a confirmation message to the user.

5.2 Sequence Diagram #2 — Read Needs + Update



Purpose

This sequence diagram illustrates how the system allows a user to view existing needs and then update one of them.

It focuses on retrieving all stored needs, displaying them to the user, accepting updated values, and saving the edited need while notifying the observers so the interface stays synchronized.

The goal is to make sure that whenever a need is updated, both the data storage and any observers are refreshed automatically.

Scenario/Feature

The user requests to read the list of needs, selects one to modify, enters new values, and the system updates the stored data.

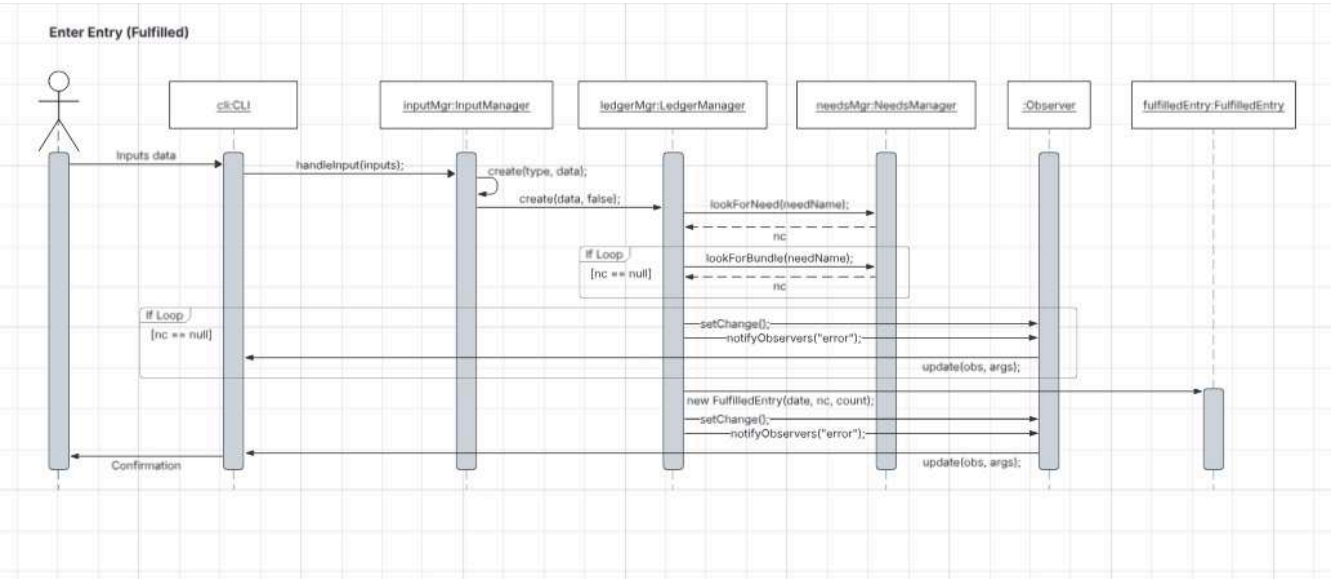
The system loads all existing needs, displays them, updates the selected need, saves the changes, and notifies observers so the user interface reflects the new state without requiring a manual refresh.

This ensures that users always interact with live, up-to-date data.

Process Explanation

1. User -> CLI
    a. The user enters a command to view needs.
2. CLI -> Input manager
    a. THE CLI calls handleOperation.
3. InputManager -> NeedsManager
    a. InputManager forwards the request by calling readALL.
4. NeedsManager -> Observer
    a. NeedsManager loads all needs from storage.
    b. Calls setChanged() and notifyObservers(needs).
5. Observer -> CLI
    a. The CLI displays the list of needs.
    b. The user selects a need and enters new updated values.

5.3 Sequence Diagram #3 — Create Entry



Purpose

This sequence diagram illustrates how the system processes a user's input to create a new *Fulfilled Entry* that records the completion of a need or bundle.

It focuses on validating the input, locating the related need or bundle, and updating the system's state to reflect fulfillment while notifying observers of the changes.

The goal is to ensure that when a fulfillment entry is recorded, all related data and observers (e.g., UI or logs) remain consistent and up to date.

Scenario/Feature

The user provides data to record a fulfilled need.

The system takes the input, identifies which need or bundle is being fulfilled, and creates a corresponding FulFilledEntry object.

If an invalid need or bundle is provided, the system notifies observers of an error.

Once the entry is successfully created, the observers are updated to reflect the change, ensuring that all dependent components (such as the user interface or reports) show the updated fulfillment status immediately.

Process Explanation

1. **User → CLI**
    a. The user inputs fulfillment data (e.g., need name, date, count).
2. **CLI → InputManager**
    a. The CLI calls handleInput to pass the entered data to the InputManager for processing.
3. **InputManager → LedgerManager**
    a. The InputManager determines the input type (fulfilled entry) and calls create(type, data) to begin creating a record.
4. **LedgerManager → NeedsManager**
    a. LedgerManager tries to find the matching need using lookForNeed(needName).
    b. If no individual need is found, it calls lookforBundle(needName) to check if the entry refers to a need bundle instead.
5. **NeedsManager → LedgerManager**
    a. The NeedsManager returns the matching needComponent if found, or null if not.
6. **LedgerManager**
    a. If no matching need or bundle exists (nc == null), LedgerManager triggers setChange() and notifyObservers("error") to signal an invalid entry.
7. **LedgerManager → FulfilledEntry**
    a. If the need or bundle is found, LedgerManager creates a new FulFilledEntry to record fulfillment.
8. **LedgerManager → Observer**
    a. After creating the entry, LedgerManager again calls setChanged() and notifyObservers to inform observers of the successful update.
9. **Observer → CLI (User)**
    The observer updates the user interface or display to confirm that the fulfillment entry was successfully recorded.
10. **User**
    Receives a confirmation message or view update showing that the need or bundle has been fulfilled.

6 Pattern Usage

6.1 Pattern #1 - Composite Pattern

The Bundle acts as a composite of BasicNeed components. Both Bundle and BasicNeed implement the NeedComponent interface, so operations like calculating total cost or checking availability can be applied uniformly to a single need or a collection of needs.

Composite Pattern	
Component	NeedComponent
Leaf	BasicNeed
Composite	Bundle
Clients	NeedsManager, LedgerManager

6.2 Pattern #2 - MVC Pattern

User input is captured by the InputManager, sent to the Model, and the View updates based on changes in the Model.

MVC Pattern	
Model	FileManager, NeedsManager, LedgerManager, Entry, FundsEntry, GoalEntry, FulfilledEntry, BasicNeed, Bundle, NeedComponent
View	CLI
Controller	InputManager

6.3 Pattern #3 - Observer Pattern

The CLI observes the LedgerManager and the NeedsManager, and when the LedgerManager or NeedsManager notifies their subscribers, the CLI updates the display or triggers appropriate actions.

Observer Pattern	
Observer	CLI
Observable	FileManager, LedgerManager, NeedsManager

7 RATIONALE

10/19/25 - Added foundational design, no major changes.

10/24/25 - Redesign the class diagram, mainly adding an InputManager and observer pattern for a better MVC pattern.

11/5/25 - Redid and improved design overview, revised onto subsystems, sequence diagrams, and pattern usage.